Engineering Report: PCB Preprocessing Pipeline Parameter Analysis

FYP Team: Semantic Aware Tiny Defect Detection

June 11, 2026

1 Executive Summary

This document outlines the technical specifications and parameter selections for the PCB RGB-to-Binary preprocessing pipeline. The objective of this algorithm is to generate high-fidelity binary masks (DeepPCB standard) from raw prototype images. This is a critical step to ensure that the VGG16 Deep-Shallow Feature Fusion model does not extract false gradient or Local Binary Pattern (LBP) features from lighting glare or background texture.

2 Stage 1: Illumination Correction (HSV Space)

To address uneven lighting and lens vignetting, the image is converted to the HSV (Hue, Saturation, Value) color space. All lighting adjustments are performed exclusively on the **V (Value)** channel.

2.1 Bilateral Filtering

```
v_filtered = cv2.bilateralFilter(v, d=9, sigmaColor=75, sigmaSpace=75)
```

- `d=9`: The diameter of the pixel neighborhood used during filtering. A value of 9 is large enough to smooth out the grainy texture of the bare PCB board without blurring the sharp edges of the copper traces.
- `sigmaColor=75` & `sigmaSpace=75`: These values determine how strictly the filter respects edges. A value of 75 means that pixels must be significantly different in color/intensity to be considered an edge. This successfully smooths the background while keeping the physical trace edges mathematically sharp for the gradient feature extractor.

2.2 CLAHE (Contrast Limited Adaptive Histogram Equalization)

```
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
```

- **clipLimit=2.0**: Sets the threshold for contrast limiting. Standard histogram equalization would amplify background noise (dust, scratches) too much. Limiting it to 2.0 gently balances the shadows caused by the magnifying lens without creating artificial defects.

- **tileGridSize=(8,8)**: Divides the image into an 8×8 grid and equalizes each block independently. This is crucial for adapting to localized glare or shadows without affecting the rest of the board.

3 Stage 2: 2D Adaptive Thresholding

```
binary_mask = cv2.adaptiveThreshold(gray_enhanced, 255,
              cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY_INV,
   45, 5)
```

This stage replaces the standard 1D Otsu method. It uses a mathematical integral image to compute local thresholds, mirroring the 2D Otsu spatial logic from recent literature.

- **Block Size = 45**: Defines the local spatial neighborhood (45×45 pixels). It must be an odd number. 45 was explicitly chosen because it is slightly larger than the thickest copper trace, ensuring the algorithm always has enough background reference to separate the trace from the board.

- **C = 5**: A constant subtracted from the computed local mean. Subtracting 5 acts as a noise buffer, preventing subtle wood-grain textures from being incorrectly segmented as copper.

4 Stage 3: Morphological Refinement

```
# Closing
kernel_close = cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3))
closed_mask = cv2.morphologyEx(binary_mask, cv2.MORPH_CLOSE,
   kernel_close, iterations=2)
# Opening
kernel_open = cv2.getStructuringElement(cv2.MORPH_RECT, (2, 2))
final = cv2.morphologyEx(closed_mask, cv2.MORPH_OPEN, kernel_open,
   iterations=1)
```

- **MORPH_CLOSE (3x3 Kernel, Iterations=2)**: Closing (Dilation followed by Erosion) bridges small gaps in the traces. The 3×3 kernel applied twice ensures that sub-pixel structural erosion caused by thresholding is repaired, presenting a continuous geometric line to the CNN.

- **MORPH_OPEN (2x2 Kernel, Iterations=1)**: Opening (Erosion followed by Dilation) erases tiny white static from the black background. A conservative 2×2 kernel ensures we do not accidentally erase true "spur" defects, which are critical diagnostic features for the model.